

计算理论导引

第2章 上下文无关文法



Contents

计算理论导引

目录

1 上下文无关文法概述

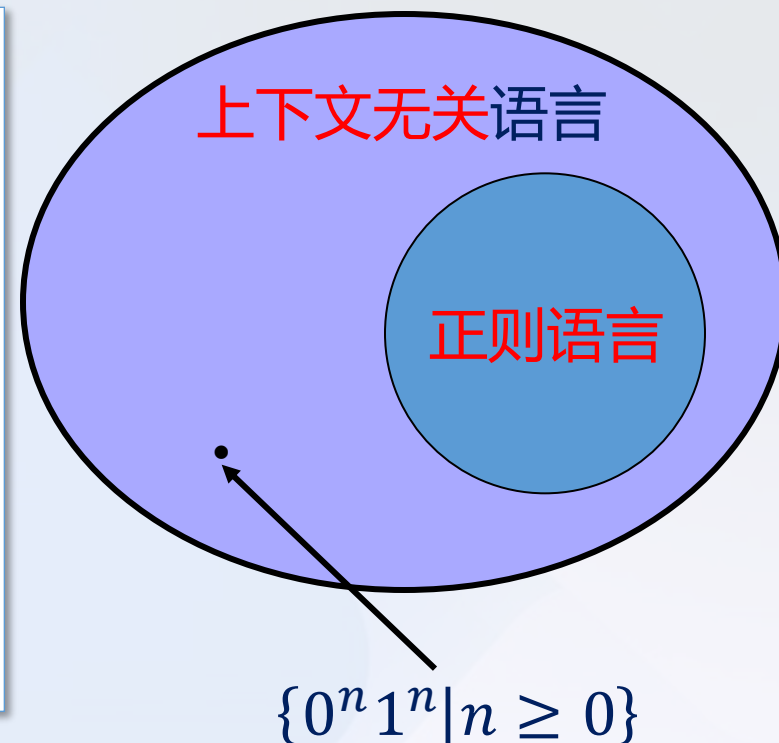
2 下推自动机

3 非上下文无关语言

4 上下文无关方法封闭性

概述

- 有穷自动机和正则表达式可以描述很多语言
- 简单语言如 $\{0^n 1^n | n \geq 0\}$ 无法描述
- 上下文无关语言包括所有的正则语言以及许多新添加的语言。
- 上下文无关文法



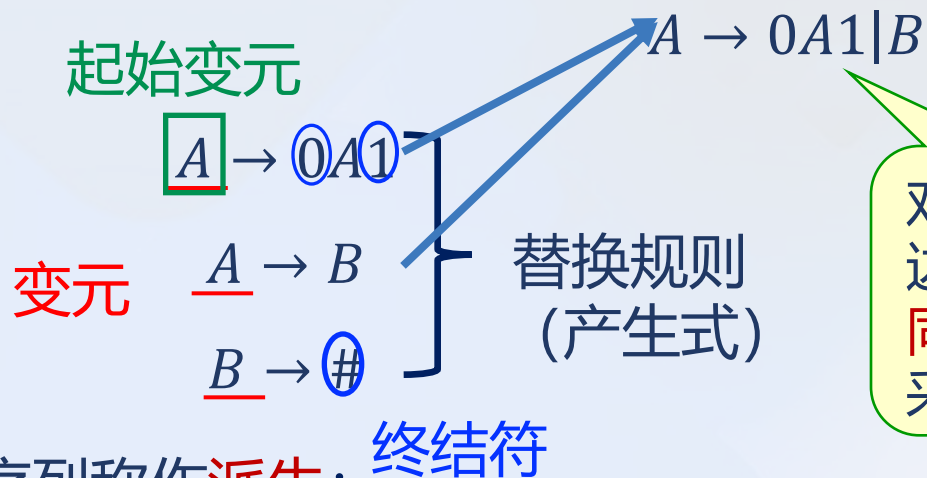
CFG 应用

- 程序设计语言的规范化及编译
- 词法分析器 → 语法分析器 (CFG) → 语义分析器

什么是上下文无关文法

示例 1

- 上下文无关文法 G_1 :



对几个左边变元相同的规则采用缩写

- 获取一个字符串的替换序列称作派生：

$$A \Rightarrow 0A1 \Rightarrow 00A11 \Rightarrow 000A111 \Rightarrow 000B111 \Rightarrow 000\#111$$
- 通过派生生成的所有的字符串构成该文法的语言。
- 用 $L\langle G_1 \rangle$ 表示文法 G_1 的语言。 $L\langle G_1 \rangle$ 是 $\{0^n \# 1^n | n \geq 0\}$ 。

能够用上下文无关文法生成的语言叫做上下文无关语言 (CFL)

思想：用 局部变化规律 + 初始条件 描述一个过程

什么是上下文无关文法

示例 1

➤ 上下文无关文法 G_1

$A \rightarrow 0A1$

$A \rightarrow B$

$B \rightarrow \#$

文法 G_1 生成字符串 000#111

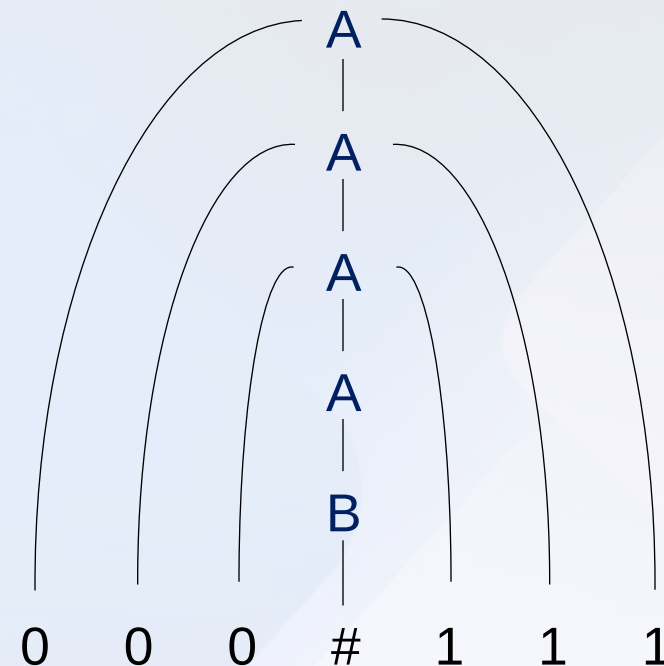
$A \Rightarrow 0A1$

$\Rightarrow 00A11$

$\Rightarrow 000A111$

$\Rightarrow 000B111$

$\Rightarrow 000\#111$



语法分析树

什么是上下文无关文法

示例 2

〈句子〉 → 〈名词短语〉〈动词短语〉
 〈名词短语〉 → 〈复合名词〉|〈复合名词〉〈介词短语〉
 〈动词短语〉 → 〈复合动词〉|〈复合动词〉〈介词短语〉
 〈介词短语〉 → 〈介词〉〈复合名词〉
 〈复合名词〉 → 〈冠词〉〈名词〉
 〈复合动词〉 → 〈动词〉|〈动词〉〈名词短语〉
 〈冠词〉 → a|the
 〈名词〉 → boy|girl|flower
 〈动词〉 → touches|likes|sees
 〈介词〉 → with

字符串: *a boy sees*

〈句子〉 → 〈名词短语〉〈动词短语〉
 → 〈复合名词〉〈动词短语〉
 → 〈冠词〉〈名词〉〈动词短语〉
 → a 〈名词〉〈动词短语〉
 → a boy 〈动词短语〉
 → a boy 〈复合动词〉
 → a boy 〈动词〉
 → a boy sees

CFG形式化定义

定义 2.1

- **上下文无关文法**是一个 4 元组 $G = (V, \Sigma, R, S)$, 这里
- a) V 是一个有穷集合, 称作**变元集**。
 - b) Σ 是一个与 V 不相交的有穷集合, 称作**终结符集**。
 - c) R 是有穷的**规则集**, 每条规则是一个变元和一个由变元和终结符组成的字符串。
 - d) $S \in V$ 是**起始变元**。

- 注: 1、在规定一个文法时, 常常只写出它的规则。
 2、出现在规则左边的所有符号都是**变元**, 其余的符号都是**终结符**。
 3、按照惯例, 起始变元是**第一条**规则左边的变元

CFG形式化定义：派生

定义 2.1

➤ 设 u, v, w 是变元和终结符的字符串, $A \rightarrow w$ 是文法的一条规则, 称 uAv **生成** uwv , 记作

$$uAv \Rightarrow uwv \quad \text{一步派生}$$

如果 $u = v$, 或存在序列 $u_1, u_2, \dots, u_k$ 使得

$$u \Rightarrow u_1 \Rightarrow u_2 \Rightarrow \dots \Rightarrow u_k \Rightarrow v$$

其中 $k \geq 0$, 则称 u **派生** (derive) v 记作

$$u \overset{*}{\Rightarrow} v \quad \text{任意步派生}$$

该文法 (生成) 的语言是:

$$\{w \in \Sigma^* \mid S \overset{*}{\Rightarrow} w\}$$

上下文无法方法举例

➤ 考虑文法 $G_4 = (V, \Sigma, R, \langle \text{EXPR} \rangle)$, 其中

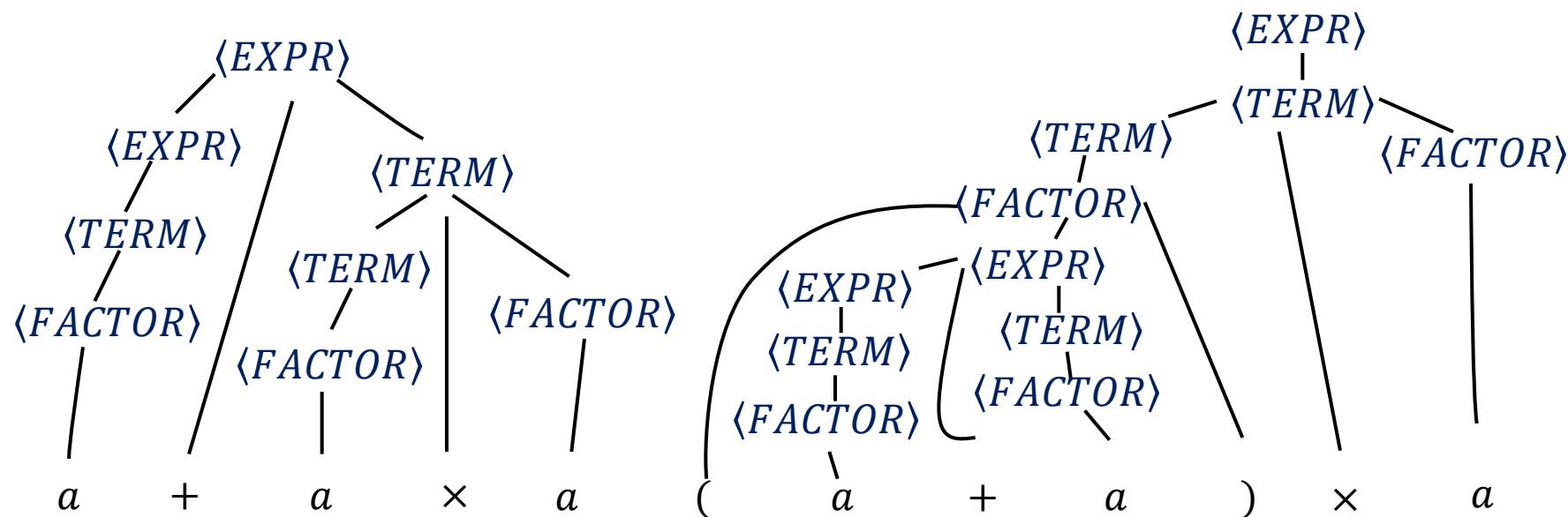
$V = \{\langle \text{EXPR} \rangle, \langle \text{TERM} \rangle, \langle \text{FACTOR} \rangle\}$, $\Sigma = \{a, +, \times, (,)\}$,

规则 R 为:

$\langle \text{EXPR} \rangle \rightarrow \langle \text{EXPR} \rangle + \langle \text{TERM} \rangle | \langle \text{TERM} \rangle$

$\langle \text{TERM} \rangle \rightarrow \langle \text{TERM} \rangle \times \langle \text{FACTOR} \rangle | \langle \text{FACTOR} \rangle$

$\langle \text{FACTOR} \rangle \rightarrow (\langle \text{EXPR} \rangle) | a$



CFG设计

1. 化繁为简

- 把1个 CFL 分为 由几个较简单的 CFL, 分别构造文法;
加入新规则 $S \rightarrow S_1 | S_2 | \dots | S_k$, 其中 S_i 是各个文法的起始变元。

举例

- 求语言 $\{0^n 1^n | n \geq 0\} \cup \{1^n 0^n | n \geq 0\}$ 的文法。
 - 1) 首先构造语言 $\{0^n 1^n | n \geq 0\}$ 的文法

$$S_1 \rightarrow 0S_1 1 | \varepsilon$$
 - 2) 其次构造语言 $\{1^n 0^n | n \geq 0\}$ 的文法

$$S_2 \rightarrow 1S_2 0 | \varepsilon$$
 - 3) 然后加上规则 $S \rightarrow S_1 | S_2$, 得到所求的文法:

$$S \rightarrow S_1 | S_2$$

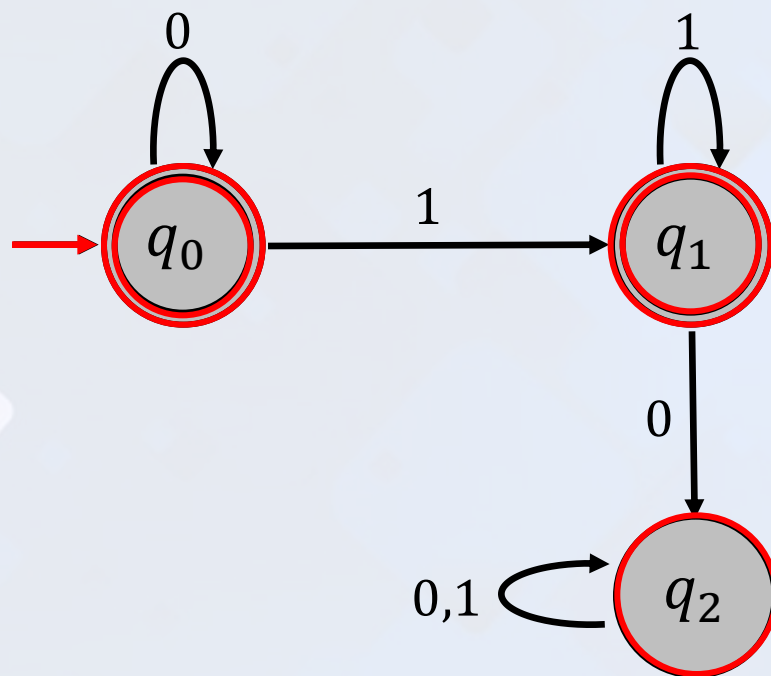
$$S_1 \rightarrow 0S_1 1 | \varepsilon$$

$$S_2 \rightarrow 1S_2 0 | \varepsilon$$

CFG 设计

2. 利用正则

- 如果一个语言是正则的，可先构造 DFA；
- 接着把 DFA 转换成等价的 CFG。



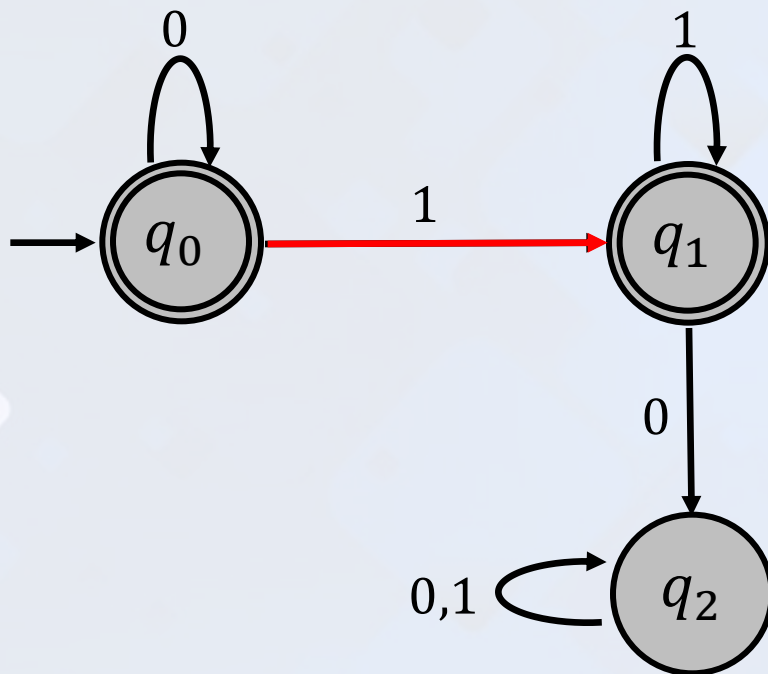
- 状态集合 $\{q_0, q_1, q_2\}$ $\Rightarrow$ 变元集合 $\{R_0, R_1, R_2\}$
- 起始状态 q_0 $\Rightarrow$ R_0 作为起始变元
- 接受状态 $\{q_0, q_1\}$ $\Rightarrow$ 添加规则 $R_0 \rightarrow \varepsilon, R_1 \rightarrow \varepsilon$

识别语言 0^*1^* 的 DFA

CFG 设计

2. 利用正则

- 如果一个语言是正则的，可先构造 DFA；
- 接着把 DFA 转换成等价的 CFG。



识别语言 0^*1^* 的 DFA

- 状态转移函数 $\delta(q_0, 1) = q_1$

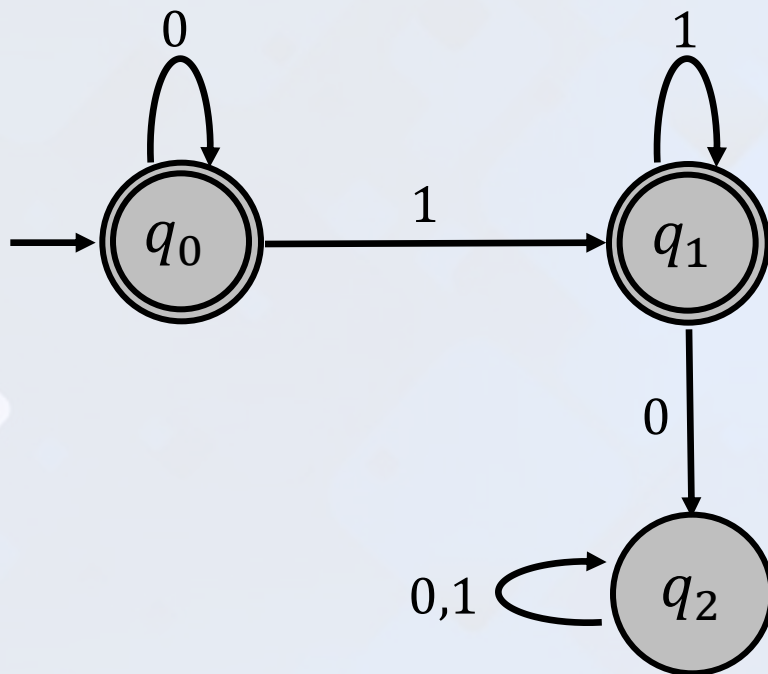
➡ 添加规则 $R_0 \rightarrow 1R_1$

- 字母表 $\{0,1\}$ ➡ 终结符 $\{0,1\}$

CFG 设计

2. 利用正则

- 如果一个语言是正则的，可先构造 DFA；
- 接着把 DFA 转换成等价的 CFG。



识别语言 0^*1^* 的 DFA

□ 由 DFA 转换为等价的 CFG:

$$R_0 \rightarrow \varepsilon | 0R_0 | 1R_1$$

$$R_1 \rightarrow \varepsilon | 0R_2 | 1R_1$$

$$R_2 \rightarrow 0R_2 | 1R_2$$

0^*1^* 的 CFG

CFG 设计

2. 利用正则

- 如果一个语言是正则的，可先构造 DFA；
- 接着把 DFA 转换成等价的 CFG。

DFA	CFG
$M = (Q, \Sigma, \delta, q_0, F)$	$G = (V, \Sigma, R, S)$
状态 q_i	添加变元 R_i
$\delta(q_i, a) = q_j$	添加规则 $R_i \rightarrow aR_j$
q_i 是接受状态	添加规则 $R_i \rightarrow \varepsilon$
q_0 是起始状态	将 R_0 作为起始变元
字母表	添加到终结符集

CFG 设计

3. 考察子串

- 针对两个“**相互联系**”的子串，识别这种语言的机器需要记住其中1个子串的信息，而这个信息是无界的。

例如： $\{0^n 1^n | n \geq 0\}$, 为了记住 0 的个数，使用以下规则

$$R \rightarrow uRv$$

产生的字符串包含 u 的部分对应包含 v 的部分。

4. 利用递归

- 字符串可能包含一定的结构，而这种结构又递归地作为另一种结构的一部分出现。此时，把**生成这种结构的变元**放在 规则中此结构对应可能出现递归的地方

歧义性举例

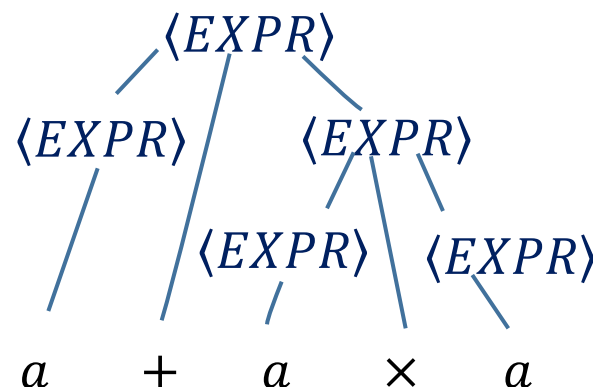
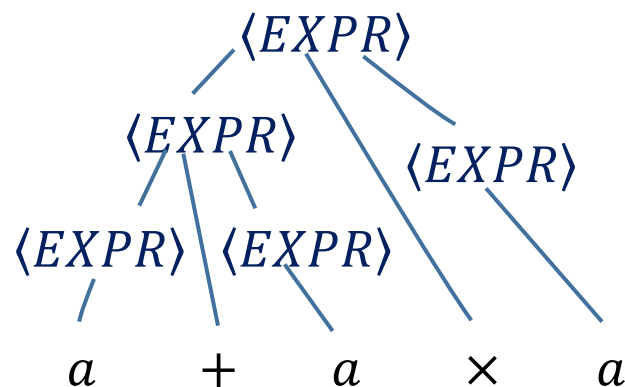
举例

下雨天留客天留我不留

1. 下雨天留客，天留我不留。
2. 下雨天留客，天留，我不留。
3. 下雨天留客，天留我？不留。
4. 下雨天留客，天留我不？留。
5. 下雨，天留客；天留我不留！
6. 下雨天，留客天，留我？不留。
7. 下雨天，留客天，留我不？留。
8. 下雨天，留客天，留，我不留？
9. 下雨，天留客，天留我不留。

歧义性

- 考虑文法 G_5

$$\langle \text{EXPR} \rangle \rightarrow \langle \text{EXPR} \rangle + \langle \text{EXPR} \rangle \mid \langle \text{EXPR} \rangle \times \langle \text{EXPR} \rangle \mid (\langle \text{EXPR} \rangle) \mid a$$


歧义性

- 如果文法以**不同的方式**产生同一个字符串，则称文法**歧义地**产生这个字符串。
- 两棵**不同语法树**，而非**两种不同派生**

歧义性

最左派生

- 定义一种以**固定次序**替换变元的派生类型。如果在每一步都是替换剩下的**最左边**的变元，则称这个派生是**最左派生**。

定义 2.4

- 如果字符串 w 在上下文无关文法 G 中有两个或两个以上不同的**最左派生**，则称在 G 中**歧义地**产生字符串 w 。如果文法 G 歧义地产生某个字符串，则称文法 G 是**歧义的**。如果一个语言只能用歧义的文法产生，这个语言叫做固有歧义语言。 $\{0^i 1^j 2^k \mid i = j \text{ 或 } j = k\}$ ， $0^n 1^n 2^n$ 只能歧义产生

乔姆斯基范式

定义 2.5

- 一个 CFG 为**乔姆斯基范式**，如果它的每个规则具有如下形式：

$$A \rightarrow BC \quad \text{一分为二}$$

$$A \rightarrow a \quad \text{终极化}$$

a 是**任意的终结符**， A 、 B 和 C 是**任意的变元**，但 B 和 C **不能是起始变元**。

此外，允许规则 $S \rightarrow \varepsilon$ ，其中 S 是**起始变元**。

定理 2.6

- **任一** CFL 都可以用**乔姆斯基范式**的 CFG 产生。

乔姆斯基范式

➤ Step 1 (加头)

添加一个新的起始变元 S_0 和规则 $S_0 \rightarrow S$ ，可保证起始变元不出现在规则的右边。（单一规则，后续还需处理）

➤ Step 2: 考虑所有的 ε 规则 (去尾)

- 1) 删除一条 ε 规则 $A \rightarrow \varepsilon$ ($A \neq S_0$)。
- 2) 对在规则右边出现的每一个 A ，删去 A 后得到一条新的规则。
- 3) 重复进行上述步骤，直至删除所有的 ε 规则（不包括起始变元）。

$$R \rightarrow uAv \longrightarrow R \rightarrow uv$$

$$R \rightarrow uAvAw \longrightarrow R \rightarrow uvAw, R \rightarrow uAvw, R \rightarrow uvw$$

$$R \rightarrow A \longrightarrow R \rightarrow \varepsilon \quad \text{除非之前已被删除过}$$

乔姆斯基范式

➤ Step 3: 处理所有的单一规则 (不允许变量一变一)

1) 删除单一规则 $A \rightarrow B$

若存在 $B \rightarrow u$, 则添加 $A \rightarrow u$, 除非其已被删除过。

2) 重复上述步骤, 直至删除所有的单一规则。

➤ Step 4.

$$A \rightarrow u_1 u_2 \cdots u_k \longrightarrow \begin{cases} A \rightarrow u_1 A_1 \\ A_1 \rightarrow u_2 A_2 \\ A_2 \rightarrow u_3 A_3 \\ \vdots \\ A_{k-2} \rightarrow u_{k-1} u_k \end{cases} \quad \begin{array}{l} \text{其中 } k \geq 3, u_i \\ \text{是变元或终结符,} \\ A_i \text{ 是新的变元。} \end{array}$$

一分为多通过多个 一分为二 实现

至此, 所有的规则只能是下述几种形式:

1) $S_0 \rightarrow \varepsilon$

2) $A \rightarrow a_i$

3) $A \rightarrow BC$

4) $A \rightarrow a_i B \longrightarrow A \rightarrow U_i B \quad U_i \rightarrow a_i$

5) $A \rightarrow B a_i \longrightarrow A \rightarrow B U_j \quad U_j \rightarrow a_j$

6) $A \rightarrow a_i a_j \longrightarrow A \rightarrow U_i U_j$

乔姆斯基范式举例

例 2.7

➤ Step 1. 运用第一步引入新起始变元得到的结果放在右边。

$$S \rightarrow ASA|aB$$

$$A \rightarrow B|S$$

$$B \rightarrow b|\epsilon$$

$$S_0 \rightarrow S$$

$$S \rightarrow ASA|aB$$

$$A \rightarrow B|S$$

$$B \rightarrow b|\epsilon$$

➤ Step 2. 左边给出删除 ϵ 规则 $B \rightarrow \epsilon$, 右边给出删除 $A \rightarrow \epsilon$ 。

$$S_0 \rightarrow S$$

$$S \rightarrow ASA|aB|a$$

$$A \rightarrow B|S|\epsilon$$

$$B \rightarrow b|\epsilon$$

$$S_0 \rightarrow S$$

$$S \rightarrow ASA|aB|a|SA|AS|S$$

$$A \rightarrow B|S|\epsilon$$

$$B \rightarrow b$$

乔姆斯基范式举例

➤ Step 3a.

左边给出删除单一规则 $S \rightarrow S$ ，右边给出删除 $S_0 \rightarrow S$ 。

$$S_0 \rightarrow S$$

$$S_0 \rightarrow \textcolor{red}{S} | ASA|aB|a|SA|AS$$

$$S \rightarrow ASA|aB|a|SA|AS|\textcolor{red}{S}$$

$$S \rightarrow ASA|aB|a|SA|AS$$

$$A \rightarrow B|S$$

$$A \rightarrow B|S$$

$$B \rightarrow b$$

$$B \rightarrow b$$

➤ Step 3b. 删除单一规则 $A \rightarrow B$ 和 $A \rightarrow S$ 。

$$S_0 \rightarrow ASA|aB|a|SA|AS$$

$$S_0 \rightarrow ASA|aB|a|SA|AS$$

$$S \rightarrow ASA|aB|a|SA|AS$$

$$S \rightarrow ASA|aB|a|SA|AS$$

$$A \rightarrow \textcolor{red}{B}|S|\textcolor{red}{b}$$

$$A \rightarrow \textcolor{red}{S}|b|\textcolor{red}{ASA}|a\textcolor{red}{B}|a|\textcolor{red}{SA}|AS$$

$$B \rightarrow b$$

$$B \rightarrow b$$

乔姆斯基范式举例

➤ Step 4.

添加新的变元和规则，把留下的所有规则转换成合适的形式。最后得到的乔姆斯基范式文法等价于 G_6 。

$$S_0 \rightarrow ASA \xrightarrow{\text{一分为二}} \begin{cases} S_0 \rightarrow AA_1 \\ A_1 \rightarrow SA \end{cases}$$

$$S \rightarrow ASA \xrightarrow{\text{一分为二}} \begin{cases} S \rightarrow AA_2 \\ A_2 \rightarrow SA \end{cases}$$

$$A \rightarrow ASA \xrightarrow{\text{一分为二}} \begin{cases} A \rightarrow AA_3 \\ A_3 \rightarrow SA \end{cases}$$

$$\begin{cases} S_0 \rightarrow aB \\ S \rightarrow aB \\ A \rightarrow aB \end{cases} \xrightarrow{\quad} \begin{cases} S_0 \rightarrow UB \\ S \rightarrow UB \\ A \rightarrow UB \\ U \rightarrow a \end{cases}$$

$$S_0 \rightarrow AA_1 | UB | a | SA | AS$$

$$S \rightarrow AA_1 | UB | a | SA | AS$$

$$A \rightarrow b | AA_1 | UB | a | SA | AS$$

$$A_1 \rightarrow SA$$

$$U \rightarrow a$$

$$B \rightarrow b$$

Contents

计算理论导引

目录

1 上下文无关文法概述

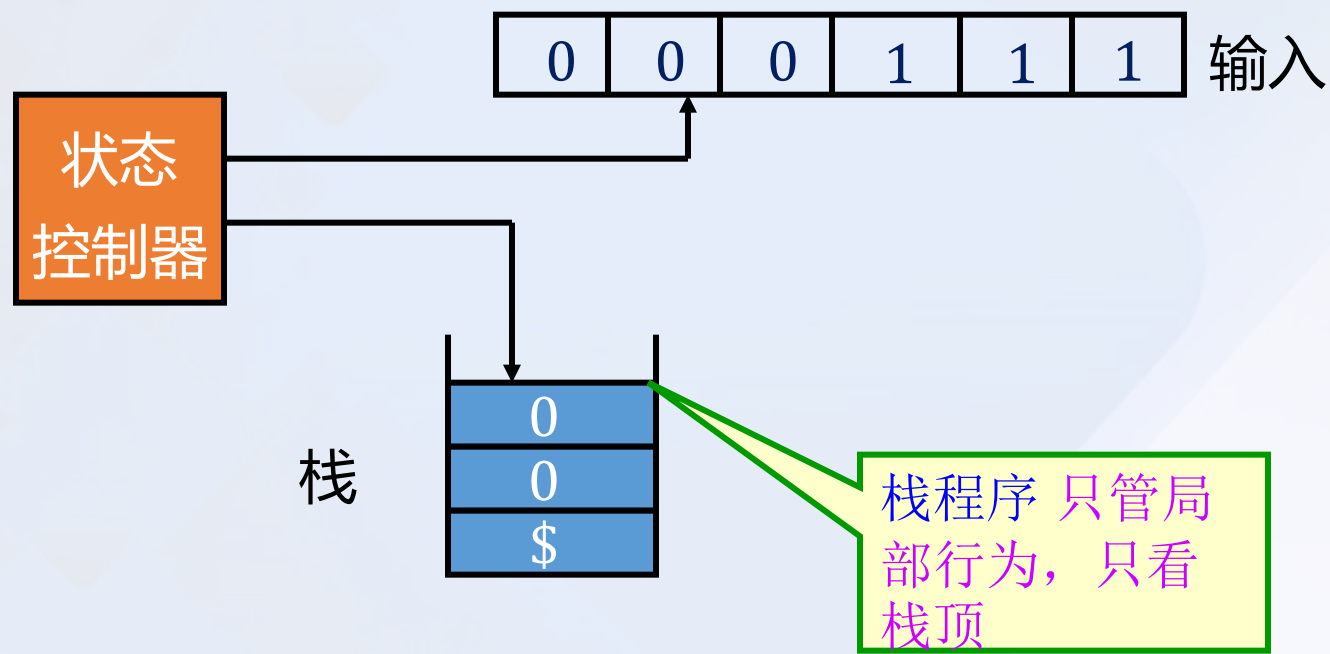
2 下推自动机

3 非上下文无关语言

4 上下文无关方法封闭性

下推自动机模型

- CFL 的语言接受模型可以看成有一个额外**栈**的 **NFA**
- 栈可以保存无限信息容量，有**推入**和**弹出**操作，先进后出
- $\{0^n 1^n | n \geq 0\}$



下推自动机形式化定义

定义 2.8

- **下推自动机**是一个 6 元组 $(Q, \Sigma, \Gamma, \delta, q_0, F)$, 这里 Q, Σ, Γ 和 F 都是有穷集合, 并且
- a) Q 是**状态集**。
 - b) Σ 是**输入字母表**。
 - c) Γ 是**栈字母表**。
 - d) $\delta: Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \rightarrow \mathcal{P}(Q \times \Gamma_\varepsilon)$ 是**转移函数**。
 - e) $q_0 \in Q$ 是起始状态。
 - f) $F \subseteq Q$ 是接受状态集。

注: $\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}, \Gamma_\varepsilon = \Gamma \cup \{\varepsilon\},$

下推自动机形式化定义

➤ $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ 的计算如下:

接收输入 $w = w_1w_2 \cdots w_m$, $w_i \in \Sigma_\varepsilon$

存在状态序列 $r_0, r_1, \cdots, r_m \in Q$

存在字符串序列 $s_0, s_1, \cdots, s_m \in \Gamma^*$, 满足下列3个条件:

(1) $r_0 = q_0$ 且 $s_0 = \varepsilon$

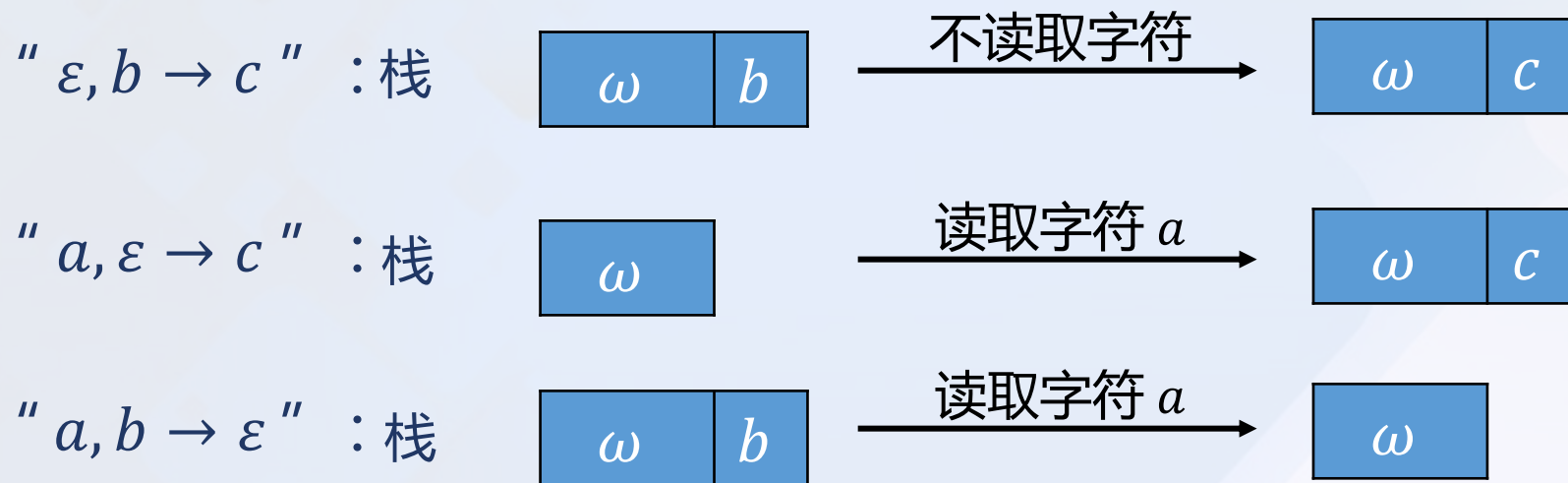
(2) $(r_{i+1}, b) \in \delta(r_i, w_{i+1}, a)$, 其中 $s_i = at$, $s_{i+1} = bt$,
这里 $a, b \in \Gamma_\varepsilon$ 和 $t \in \Gamma^*$, $0 \leq i \leq m-1$,

注: M 在每一步都完全按照当时的状态, 栈顶符号和下一个输入符号动作。

(3) $r_m \in F$, 说明在输入结束时出现一个接受状态
则 s_i 是 M 接受分支中的栈内容序列。

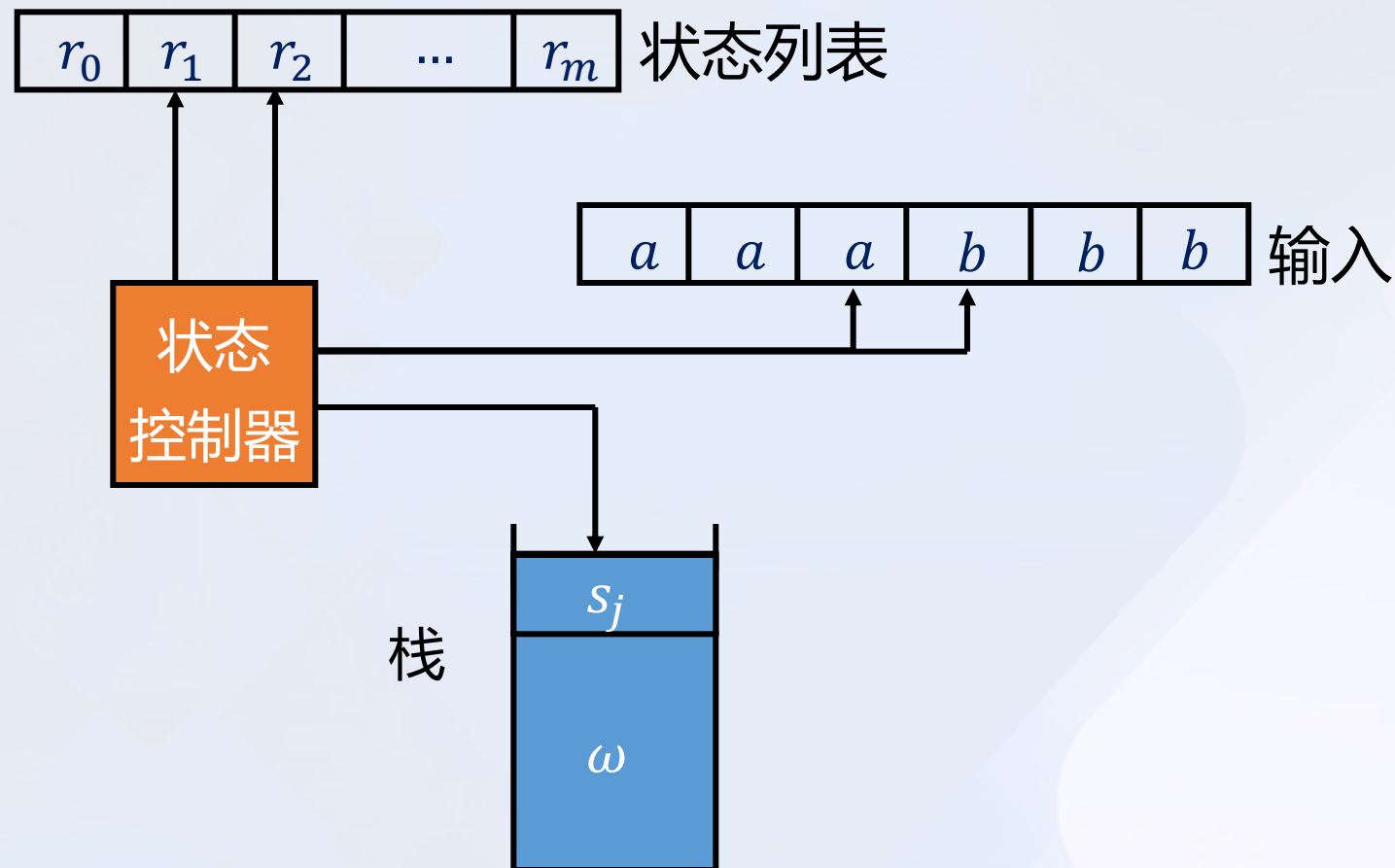
下推自动机举例

- 可以用状态图来描述 PDA。用 " $a, b \rightarrow c$ " 表示当机器从输入中读到 a 时可以用 c 替换栈顶的符号 b 。



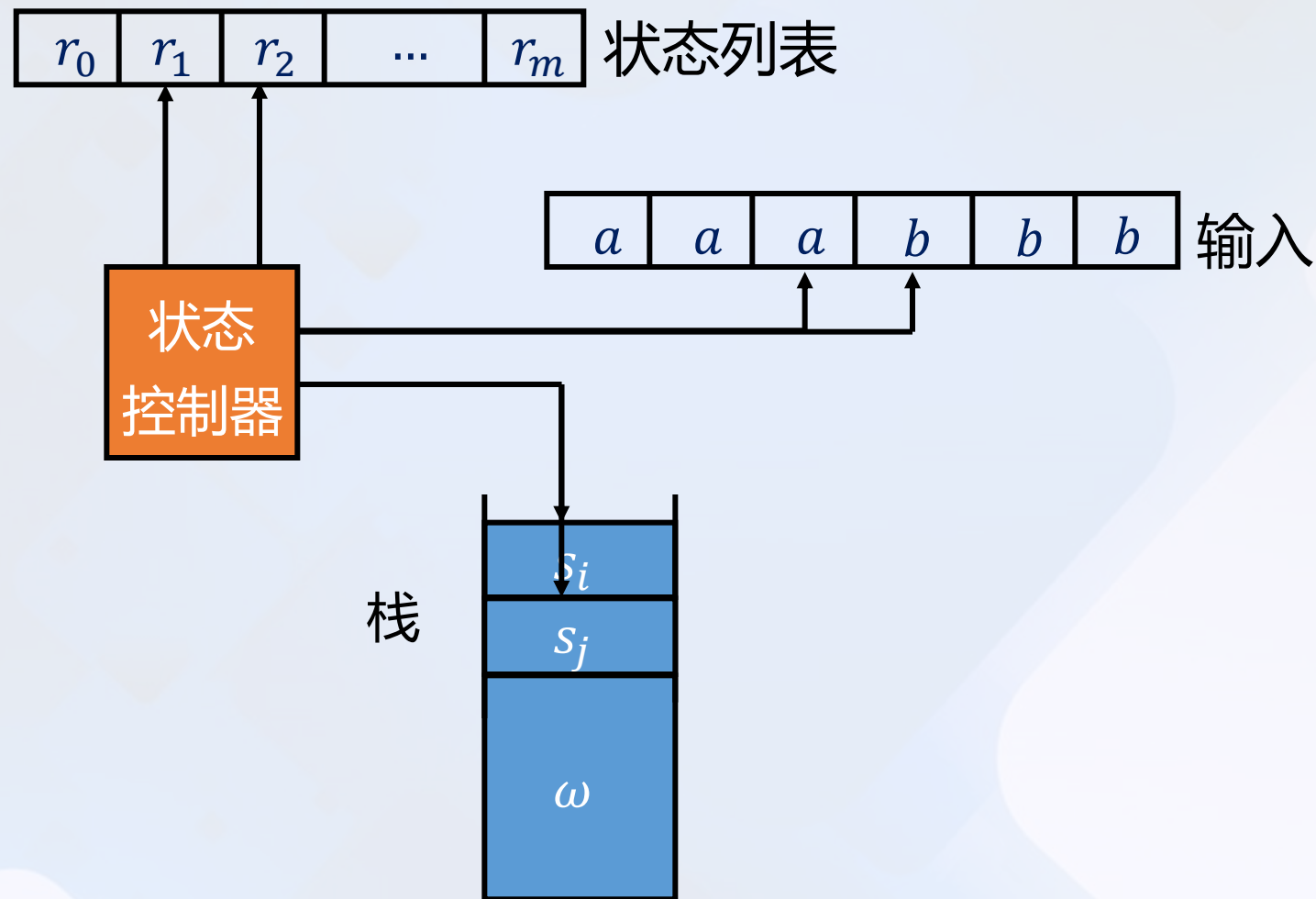
下推自动机模型

$$(r_2, s_j) \in \delta(r_1, a, s_i)$$



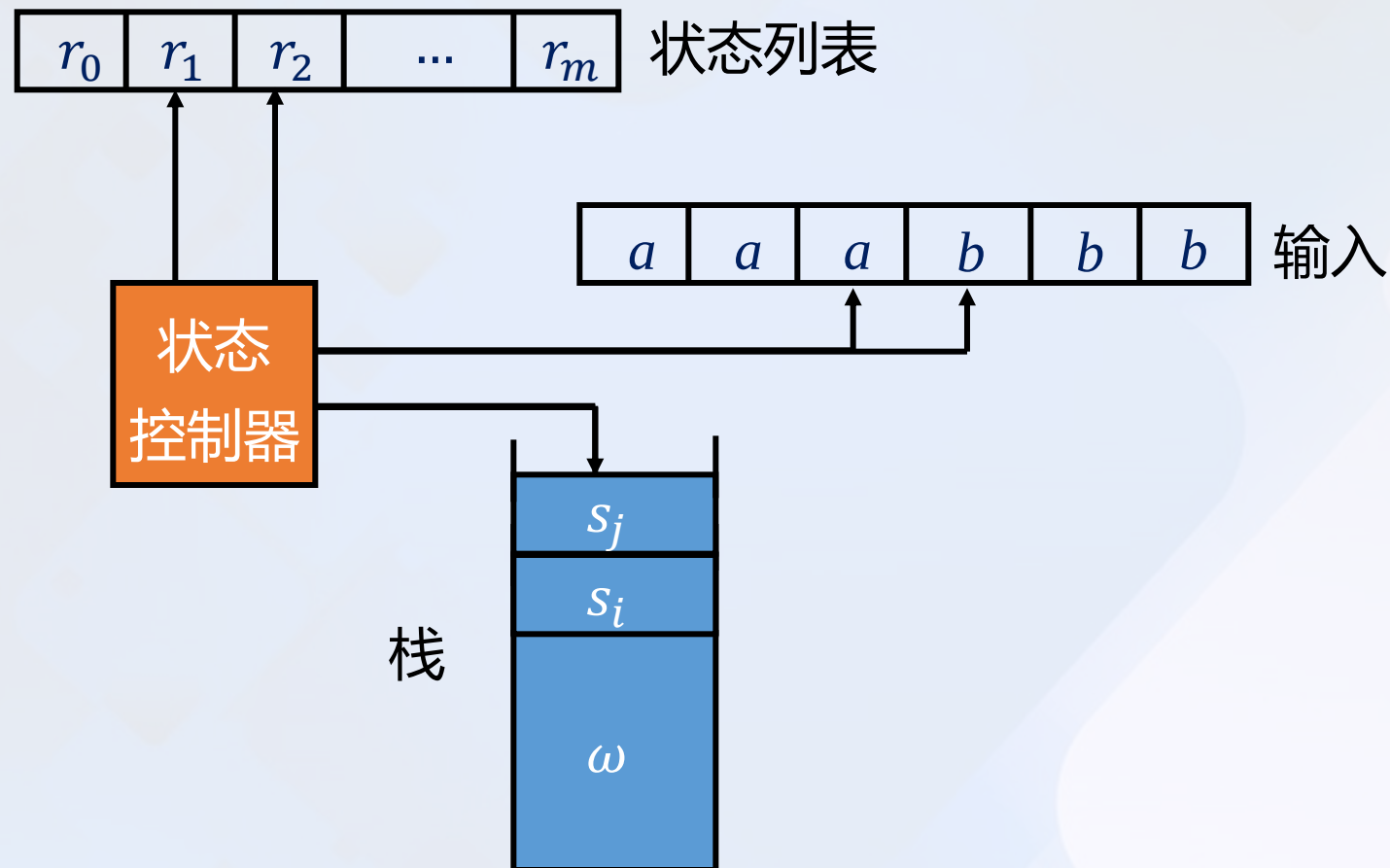
下推自动机模型

$$(r_2, \varepsilon) \in \delta(r_1, a, s_i)$$



下推自动机模型

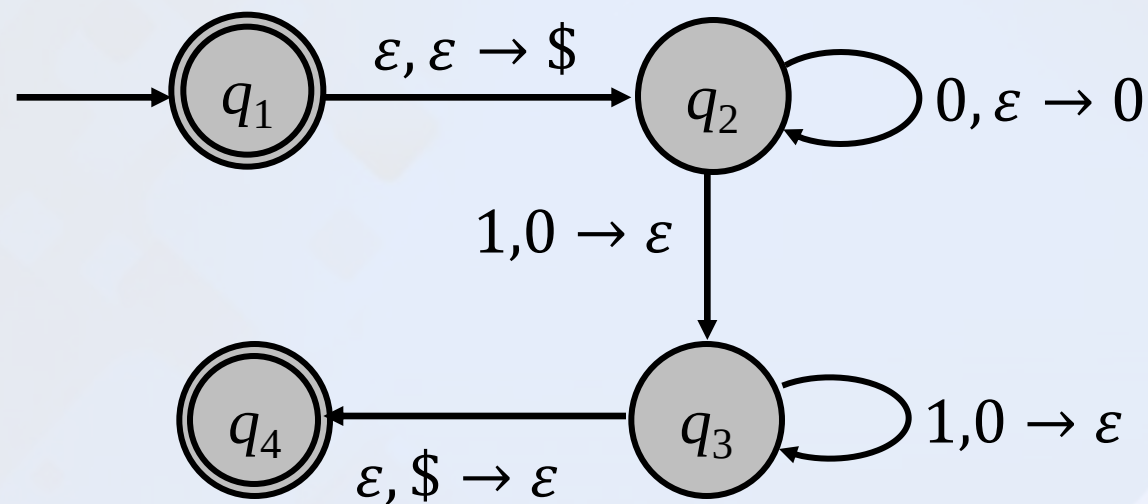
$$(r_2, s_j) \in \delta(r_1, a, \varepsilon)$$



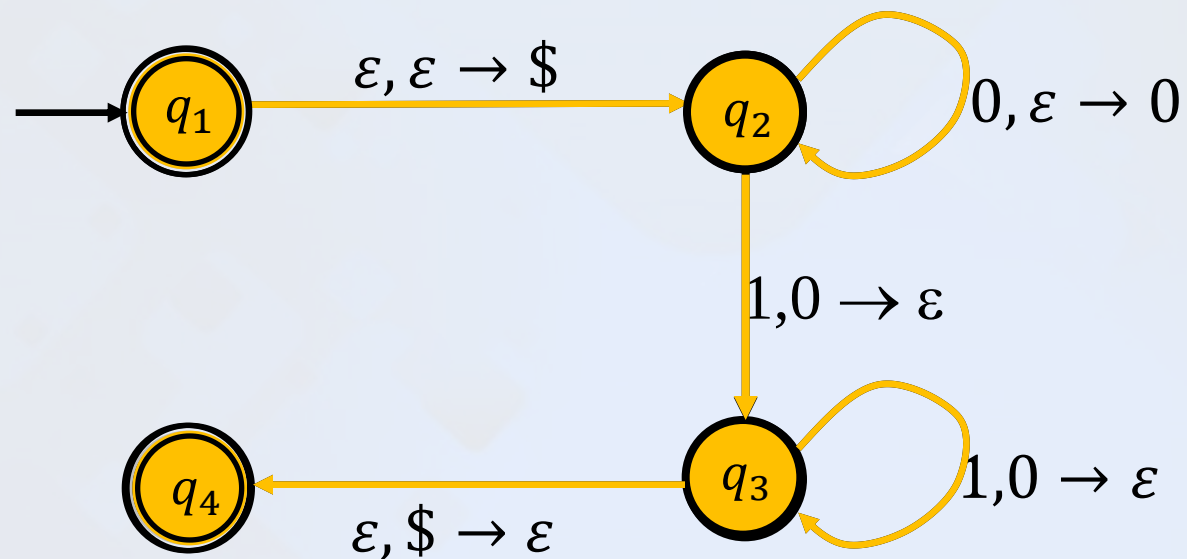
下推自动机举例

例 2.9

➤ 下面是语言 $\{0^n 1^n | n \geq 0\}$ 的 PDA 的形式定义。



下推自动机工作方法



接受

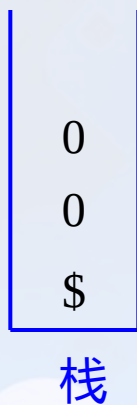
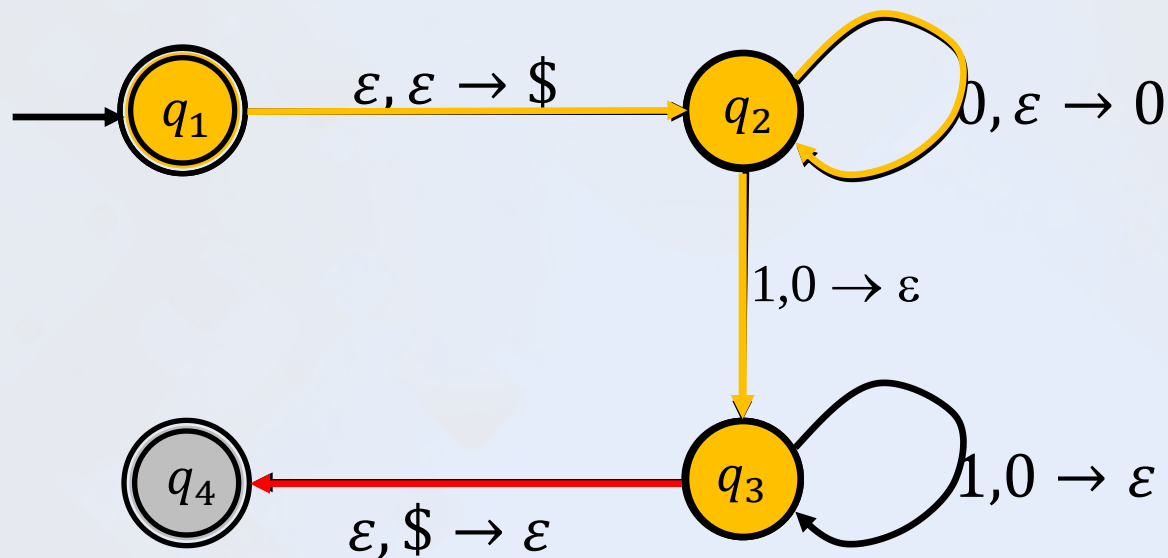
0
0
0
\$

Stack

0 0 0 1 1 1

Input

下推自动机工作方法



\$不在栈顶，无法弹出！

拒绝

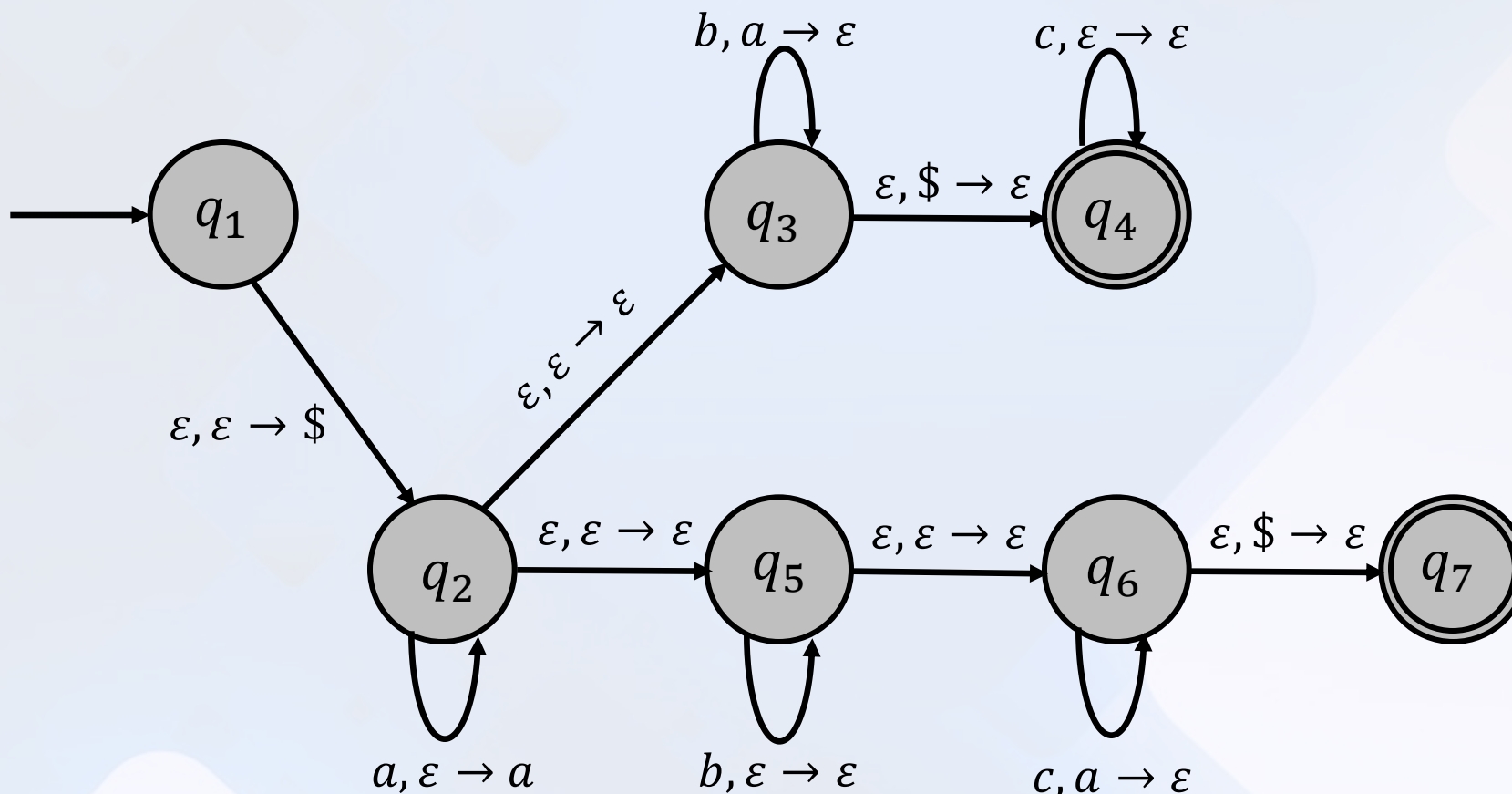


构造下推自动机例题

例 2.10

➤ 给出一台识别下述语言的PDA:

$$\{a^i b^j c^k \mid i, j, k \geq 0 \text{ 且 } i = j \text{ 或 } i = k\}$$

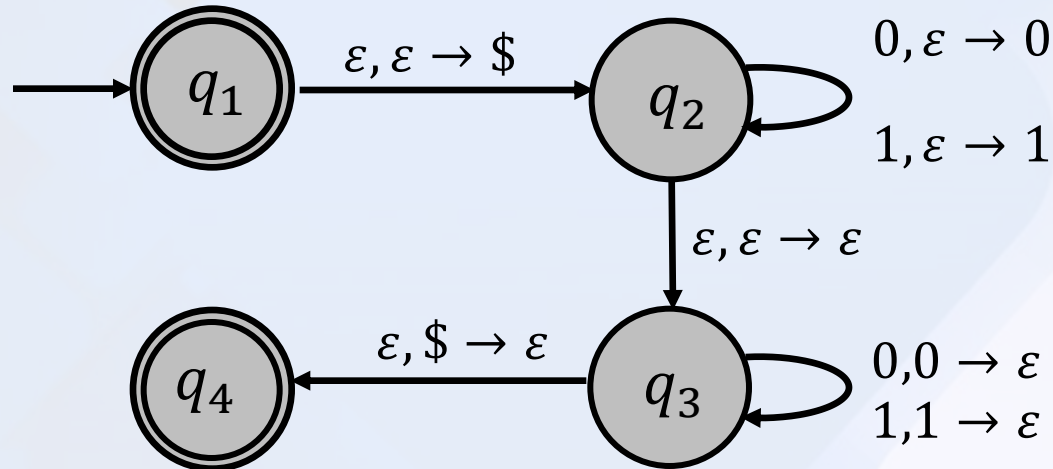


构造下推自动机例题

例 2.11

➤ 给出一台识别下述语言的PDA (w^R 表示倒写的 w):

$$\{ww^R \mid w \in \{0, 1\}^*\}$$



总结

定义 2.8

➤ 概念

- a) 上下文无关文法 (CFG)
- b) 上下文无关语言 (CFL)
- c) 派生、最左派生、固有歧义性
- d) 乔姆斯基范式 (CNF)
- e) 下推自动机 (PDA)

➤ 方法

- a) 设计上下文无关文法 (CFG)、设计下推自动机 (PDA)
- b) 求等价乔姆斯基范式 (CNF)